SAÉ6.IOM.01

Rapport de projet

28 mars 2026

Rémi MOULIN
Étudiant en Réseaux et Télécommunications

Professeur : **Xavier JEANNERET**

Table des matières

I. Introduction 4
II. Cahier des charges 5
1. Fonctionnalités attendues 5
2. Contraintes techniques 5
3. Technologies susceptibles d'être utilisées 5
Architecture Matérielle (Hardware) 5
4. Architecture Logicielle et Protocoles 5
III. Justification des choix techniques 7
1. ESP32 comme microcontrôleur central 7
2. Vis sans fin pour la distribution 7
3. Cellule de pesage HX711 pour la mesure 7
4. RFID pour l'identification des animaux 7
5. Capteur PIR pour la détection de présence 7
6. Capteur ultrasonique HC-SR04 pour le stock 8
7. MQTT et Raspberry Pi 4 pour la supervision 8
8. Flutter pour l'application mobile 8
IV. Conception mécanique 9
1. Modélisation 3D avec Shapr3D 9
a) Choix du logiciel 9
b) Vue d'ensemble du distributeur 9
2. Les pièces du distributeur 10
a) La base en deux parties 10
b) Le bloc électronique 10
c) Le bloc de distribution et la vis sans fin 10
d) La colonne et le réservoir de stockage 11
e) Le couvercle en TPU 12
f) La gamelle amovible et les pièces secondaires 12
3. Impression 3D 13
a) Imprimantes et matériaux utilisés 13
b) L'évolution de la vis sans fin 13
c) Le choix du PLA transparent pour le réservoir 13
d) Compatibilité alimentaire 14
V. Conception électronique 15
1. Choix des composants 15

2. Schéma de câblage avec Fritzing 15
3. Brochage de l'ESP32 16
4. Fabrication du PCB 16
a) Logiciels utilisés : FlatCAM et Candle 16
b) La CNC 3018 3 axes 17
c) Problèmes rencontrés 18
VI. Programmation ESP32 19
1. Architecture logicielle 19
a) Structure du code 19
b) Bibliothèques utilisées 19
2. Communication BLE 19
a) Protocole JSON 19
b) Commandes implémentées 19
c) Fragmentation des messages 19
3. Distribution contrôlée par pesée 20
a) Principe de fonctionnement 20
b) Calibration de la balance 20
c) Seuil d'arrêt et confirmations anti-vibrations 20
d) Problèmes de précision et solutions 20
4. Identification RFID 21
5. Mesure du niveau de stock 21
6. Détection de présence 21
7. Retour visuel LED 21
8. Communication réseau 21
a) WiFi et reconnexion automatique 21
b) MQTT : publication périodique et topics 21
VII. Application mobile Flutter 23
1. Technologies et architecture 23
2. Page Scan BLE 23
3. Page Liste des animaux 24
a) Requête unique get_all 24
b) Barre de statut 24
4. Page Ajout animal 25
a) Formulaire avec recommandations automatiques 25
b) Association du badge RFID 26

5. Page Paramètres 26
VIII. Infrastructure serveur 28
1. Architecture globale 28
2. Mosquitto 28
3. Node-RED 28
4. MariaDB 28
5. Grafana 29
IX. Problèmes rencontrés et solutions 30
1. BLE invisible au scan 30
2. Surdosage lors de la distribution 30
3. La vis sans fin : 4 impressions 30
4. Crash de la CNC 30
5. Listeners BLE concurrents dans Flutter 31
6. Filtre de scan BLE invisible sur Android 31
Conclusion 32
Glossaire 33
Bibliographie 34
Annexes 35
A. Notice d'utilisation 35
B. Compilation de l'application mobile 35
C. Notice de mise à jour et dépannage 35
D. Lien GitHub 36
Table des illustrations 37

I. Introduction

Dans le cadre de ma formation en BUT Réseaux et Télécommunications, parcours Internet des Objets et Mobilité, à l'IUT de Blois, j'ai réalisé un projet de SAE portant sur la conception d'un distributeur automatique de nourriture pour animaux. Le sujet m'a été imposé et correspond au sujet n°6 de la SAÉ6.IOM.01 de l'année 2026.

L'objectif principal est de concevoir un système complet capable d'identifier un animal, de distribuer automatiquement une quantité précise de nourriture en fonction de ses besoins, et de suivre la consommation en temps réel sur un home assistant. Le distributeur peut gérer jusqu'à trois animaux différents, chacun avec sa propre ration et son propre temps d'attente entre les repas.

Ce rapport présente d'abord le cahier des charges avec les fonctionnalités attendues, puis la justification des choix techniques, ensuite la réalisation technique dans toutes ses dimensions, et enfin les problèmes rencontrés et les solutions apportées. L'ensemble du code source est disponible sur le dépôt GitHub du projet. (Lien en annexe)

II. Cahier des charges

Le cahier des charges techniques a été rédigé en début de projet et correspond aux livrables 1 à 4 de la SAÉ. Le sujet imposé demande la conception d'un distributeur programmable de nourriture pour animaux avec détection de présence et mesure de la quantité distribuée et prélevée.

1. Fonctionnalités attendues

Le système doit répondre aux exigences fonctionnelles suivantes :

- Mesure précise de la quantité distribuée et consommée
- Collecte des données avec historique et statistiques de consommation
- Programmation des cycles de nourrissage
- Horodatage de la présence de l'animal
- Alertes si l'animal n'a pas mangé au cours d'une période choisie
- Possibilité de déclenchement par BLE

2. Contraintes techniques

La ration par repas doit être configurable entre 10 et 200 grammes. Le temps d'attente entre deux repas pour un même animal peut aller de 1 heures à 12 heures. La distribution doit être précise, ce qui demande un système de pesée en continu. Le distributeur doit fonctionner même sans connexion WiFi ou MQTT. Le système doit gérer plusieurs animaux en même temps.

3. Technologies susceptibles d'être utilisées

Pour répondre à ces besoins tout en garantissant la modularité et l'isolation entre les parties alimentaires et électroniques, les technologies suivantes sont retenues :

Architecture Matérielle (Hardware)

- **Cœur du système :** Microcontrôleur ESP32
- **Pesage :** Capteur de pesée HX711
- **Détection :** RFID et PIR
- **Mesure de stock :** Capteur ultrason HC-SR04
- **Information visuelle :** LED 3 couleurs KY016 (j'ai fait le choix de ne pas l'inclure dans le projet final, car cette dernière n'importait pas beaucoup d'information, le code est tout de même prévu pour l'utiliser)

4. Architecture Logicielle et Protocoles

- **Communication de supervision :** Protocole MQTT. L'ESP32 agira en tant que "Client" publiant des messages (poids, alertes) vers un "Broker" installé sur un

Raspberry Pi 4. Ce choix assure une transmission fluide et robuste vers la tablette de suivi global.

- **Configuration locale :** Protocole Bluetooth Low Energy (BLE). Il sera utilisé pour l'interface de proximité, permettant de paramétrer les profils animaux (poids, espèce, rations personnalisées) sans dépendre du réseau Wi-Fi.
- **Interface Mobile :** Création d'une application de réglage, avec Flutter pour générer rapidement une interface BLE sans nécessiter une connaissance du développement d'applications mobile. Compatible Android et IOS

III. Justification des choix techniques

Cette section explique pourquoi chaque composant et chaque technologie a été choisi pour répondre aux fonctionnalités demandées dans le cahier des charges.

1. ESP32 comme microcontrôleur central

Le cahier des charges demande une communication BLE pour le déclenchement et la configuration, ainsi qu'une connexion WiFi pour la remontée de données vers un serveur. L'ESP32 intègre nativement le WiFi et le Bluetooth Low Energy, ce qui évite d'ajouter des modules externes. Il dispose aussi de suffisamment de GPIO pour connecter tous les capteurs nécessaires.

2. Vis sans fin pour la distribution

Le cahier des charges exige une mesure précise de la quantité distribuée. Contrairement à une trappe qui libère tout d'un coup, la vis sans fin délivre une quantité progressive par rotation. Cela permet de doser finement et d'arrêter la distribution dès que le poids voulu est atteint. La rotation en spirale exerce aussi une force constante qui repousse les aliments vers la sortie, ce qui réduit les risques de blocage.

3. Cellule de pesage HX711 pour la mesure

La fonctionnalité de mesure précise demande un capteur capable de mesurer le poids dans la gamelle en temps réel. Le module HX711 avec une cellule de pesage 1 kg offre une précision de plus ou moins 0,2 grammes, ce qui est largement suffisant pour des croquettes de 1 à 3 grammes. La consommation réelle est calculée par la différence entre ce qui a été distribué et ce qui reste dans la gamelle.

4. RFID pour l'identification des animaux

Le cahier des charges demande un horodatage de la présence de l'animal et des alertes en cas d'absence de consommation. Pour savoir quel animal est présent, il faut pouvoir l'identifier de manière unique. Le badge RFID, porté au collier de l'animal, permet cette identification sans contact. Chaque badge a un UID unique, ce qui permet de distinguer les animaux et d'enregistrer l'heure de chaque passage.

5. Capteur PIR pour la détection de présence

Le lecteur RFID consomme de l'énergie et du temps processeur s'il est actif en permanence. Le capteur PIR détecte un mouvement dans un rayon de 2 mètres et sert de déclencheur : le RFID ne s'active que quand une présence est détectée. Cela évite aussi les déclenchements accidentels quand personne n'est devant le distributeur.

6. Capteur ultrasonique HC-SR04 pour le stock

La collecte de données avec historique passe aussi par le suivi du niveau de stock. Le capteur ultrasonique, placé en haut du réservoir, mesure la distance jusqu'aux croquettes. Plus la distance est petite, plus le réservoir est rempli. Cette donnée est remontée au serveur pour générer des alertes quand le stock est bas.

7. MQTT et Raspberry Pi 4 pour la supervision

La collecte de données et les alertes d'absence demandent un serveur capable de stocker l'historique et d'afficher des statistiques. Le protocole MQTT est léger et adapté à l'IoT. Le Raspberry Pi 4 héberge le broker Mosquitto, Node-RED pour le traitement des données, MariaDB pour le stockage, et Grafana pour la visualisation. L'alerte d'absence est configurée directement dans Grafana avec un seuil modifiable.

8. Flutter pour l'application mobile

Le cahier des charges demande une possibilité de déclenchement par BLE et la configuration des profils animaux. Je ne souhaiter pas faire un simple bouton pour remplir cette condition. Je me suis donc tourné vers Flutter un framework de Google qui permet de créer une application Android avec un seul code source, compatible aussi iOS. La bibliothèque flutter_blue_plus gère la communication BLE.

IV. Conception mécanique

1. Modélisation 3D avec Shapr3D

a) Choix du logiciel

J'ai choisi Shapr3D pour modéliser l'ensemble des pièces du distributeur. Je l'avais déjà utilisé dans des projets précédents (notamment en lien avec la Fabrique connectée) donc je le maitriser déjà un petit peu. Le logiciel permet des exports en STL, OBJ, 3MF ce qui est compatibles directement avec les slicers comme Bambu Studio.

b) Vue d'ensemble du distributeur

Le distributeur est composé de 9 pièces principales et de plusieurs petites pièces complémentaires. L'ensemble représente environ 1,5 kg de PLA imprimé, ce qui a demandé plusieurs jours d'impression. La majorité des pièces sont passées du premier coup, après un travail de ponçage pour les finitions. Le distributeur s'organise verticalement : la base en bas qui accueille la gamelle et l'électronique, le bloc de distribution avec la vis sans fin au milieu, et le réservoir de stockage en haut.

Figure 1 : Distributeur assemblé complet

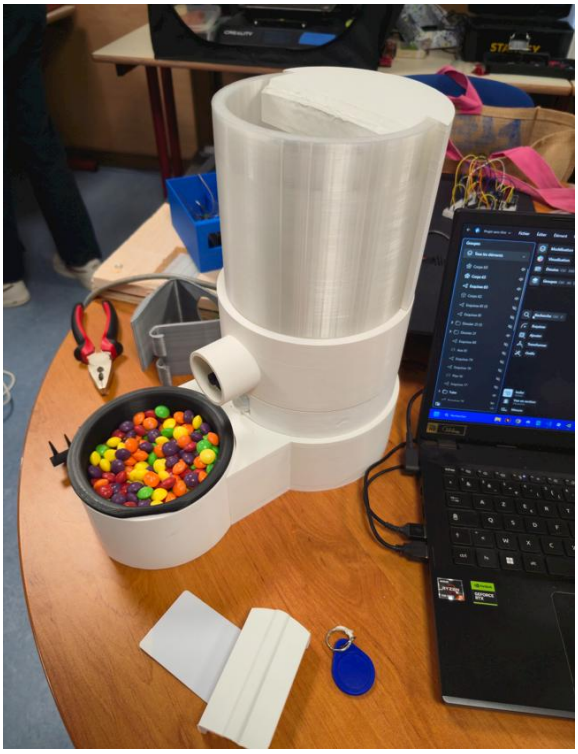

Figure 2 : Distributeur avec des Skittles pour les tests

2. Les pièces du distributeur

a) La base en deux parties

La base du distributeur est divisée en deux parties car la pièce complète était trop grande pour être imprimée en une seule fois. La première partie accueille la colonne centrale du distributeur, et la seconde sert de support pour la gamelle. Les deux parties s'emboîtent. Cette séparation a été pensée juste avant le moment d'imprimer où j'ai réalisé les limites du plateau d'impression.

b) Le bloc électronique

Le bloc électronique est la partie où se trouve la carte et le capteur PIR. J'ai conçu un système de verrouillage à trois crans inspirés d'un tutoriel YouTube. Le principe est simple : trois pins descendent dans des encoches et on fait tourner le bloc pour le verrouiller en position. Ce système permet d'ouvrir facilement le bloc pour accéder à l'électronique sans outils. Un trou à l'arrière permet de faire passer un câble micro-USB pour l'alimentation.

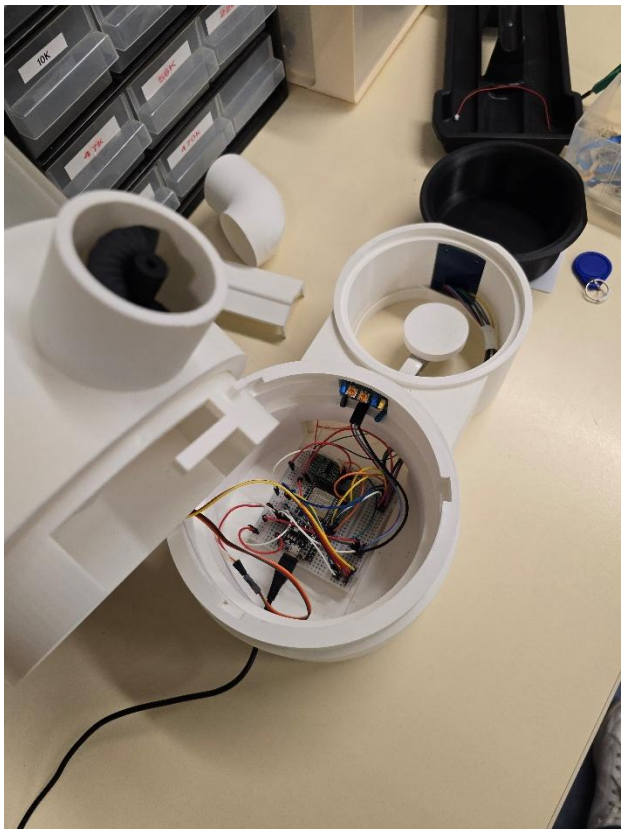

Figure 3 : Bloc électronique ouvert avec la breadboard

c) Le bloc de distribution et la vis sans fin

Le bloc de distribution se situe au-dessus du bloc électronique. Il contient un tuyau dans lequel tourne la vis sans fin, avec une pente pour que les croquettes glissent

naturellement vers la sortie. La vis sans fin est entraînée par le servomoteur continu et permet de doser la quantité de croquettes distribuées.

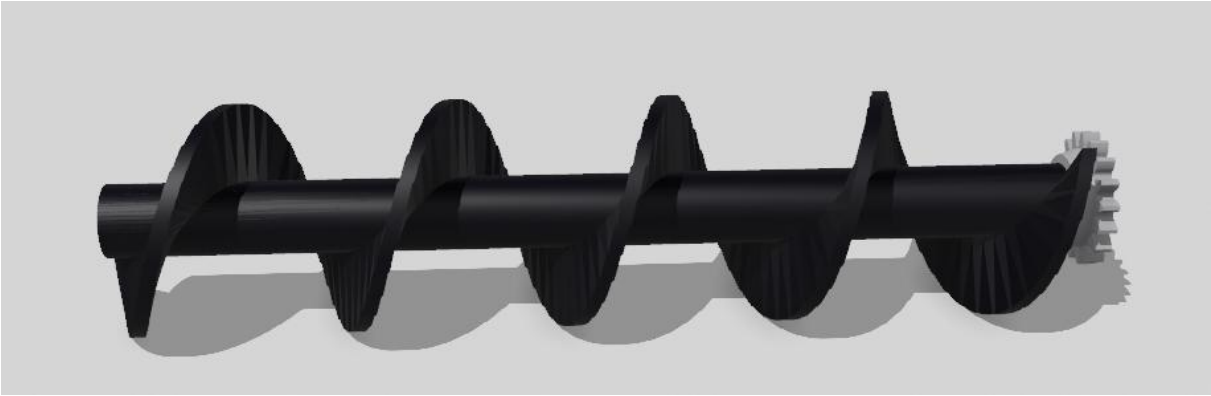

Figure 4 : Modèle 3D de la vis sans fin

Figure 5 : Vue intérieure du stockage et de la vis sans fin

d) La colonne et le réservoir de stockage

Au-dessus du bloc de distribution se trouvent deux éléments : une colonne qui sert de passage pour les câbles et qui accueille le capteur ultrason en haut, positionné au-dessus du stockage pour mesurer le niveau. Et le réservoir de stockage lui-même. Le réservoir a été imprimé en PLA transparent pour pouvoir voir le niveau de croquettes à l'œil nu. Le rendu final n'est pas vraiment transparent mais plutôt opaque à cause des couches d'impression. Le rendu reste suffisant pour voir le niveau des croquettes.

Figure 6 : Colonne avec le capteur ultrason

e) Le couvercle en TPU

Le couvercle du réservoir a été imprimé en TPU, sur l'Ultimaker Connect 2+. L'idée était d'avoir un couvercle flexible qui s'emboîte facilement sur le réservoir. Je me suis inspirée du distributeur (non automatique) de mon chat qui utiliser le même système. Le résultat est un peu décevant car le couvercle a pris toutes les imperfections du plateau en verre de l'Ultimaker qui est très abîmé. Malgré ça ma première impression en TPU c'est plutôt bien passer (ce qui est assez rare pour une première fois avec du TPU) et le couvercle remplit bien sa fonction.

f) La gamelle amovible et les pièces secondaires

La gamelle est imprimée en PLA et est amovible pour pouvoir être nettoyée facilement. Elle se pose simplement sur le support de la base et peut être retirée à tout moment. En plus des pièces principales, j'ai modélisé et imprimé plusieurs petites pièces : un bloc pour verrouiller la position du servomoteur, un petit plateau pour augmenter la surface de la cellule de pesée qui est une barre, et un petit tuyau pour mieux maintenir la vis sans fin et empêcher le chat de faire tomber des croquettes en grattant le tuyau.

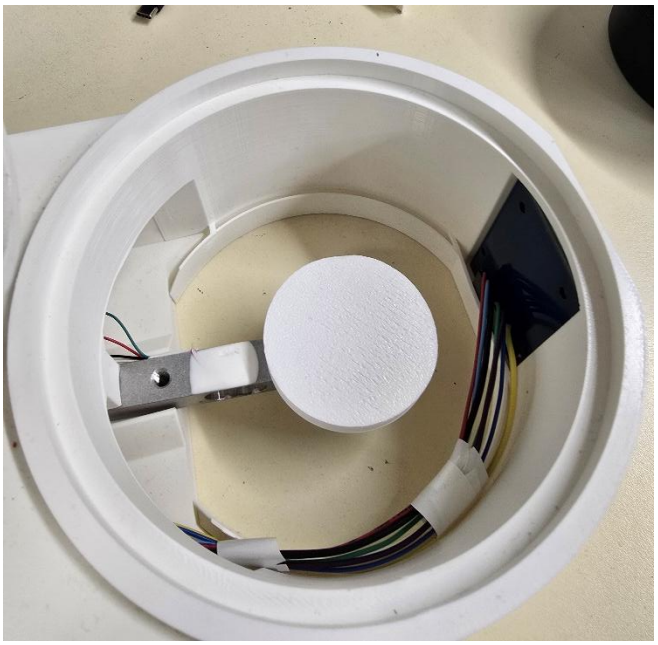

Figure 7 : Partie pesée avec la cellule de charge

3. Impression 3D

a) Imprimantes et matériaux utilisés

J'ai utilisé trois imprimantes différentes au cours du projet. La majorité des pièces ont été imprimées sur la Bambu Lab A1 Mini et la Bambu Lab X1 Carbon en PLA. Le couvercle en TPU a été imprimé sur l'Ultimaker Connect 2+. L'ensemble représente environ 1,5 kg de filament PLA et a demandé plusieurs jours d'impression au total.

b) L'évolution de la vis sans fin

La vis sans fin a été la pièce la plus difficile à imprimer. Il m'a fallu quatre essais avant d'obtenir un résultat correct.

Le premier essai a été fait en PETG. Les supports étaient impossibles à enlever proprement, ce qui rendait la pièce inutilisable. Le deuxième essai a été fait en PLA, mais les supports étaient toujours très difficiles à retirer et la vis était trop fine. Pour le troisième essai, j'ai utilisé du PLA support, pour résoudre le problème des supports. La vis était bien imprimée mais les creux étaient trop réguliers, ce qui provoquait des blocages de croquettes. Pour le quatrième et dernier essai, j'ai espacé davantage les creux et rendu la vis plus fine, toujours avec du PLA support. Le résultat était parfait, les croquettes passaient sans blocage.

c) Le choix du PLA transparent pour le réservoir

Pour le réservoir de stockage, j'ai utilisé du PLA transparent avec des paramètres spécifiques : seulement 2 couches extérieures et un remplissage en lignes droites dans le sens horizontal. Ce choix permet d'obtenir une certaine transparence sur les

côtés du réservoir pour voir le niveau de croquettes. J'ai choisi de n'utiliser que 2 couches extérieures pour essayer de maximiser la transparence sur les côtés. J'aurais pu augmenter la transparence en réduisant le remplissage, mais j'aurais perdu en résistance ce que je ne voulais pas.

d) Compatibilité alimentaire

Le distributeur est une maquette et le PLA n'est pas idéal pour le contact alimentaire dans ce contexte. Même si le PLA d'origine végétale, les buses d'impression peuvent libérer des traces de métaux, et la surface des pièces imprimées favorise le développement bactérien. Pour un produit destiné à un usage réel, il faudrait utiliser des buses en acier inoxydable, un filament alimentaire comme du PETG food-safe, et appliquer un revêtement alimentaire sur les surfaces en contact avec la nourriture.

V. Conception électronique

1. Choix des composants

Petit rappel sur les choix des composants, l'ESP32 a été choisi comme microcontrôleur central car il intègre le WiFi et le Bluetooth Low Energy, ce qui est essentiel pour la communication avec l'application mobile et le serveur MQTT. Le lecteur RFID MFRC522 communique en SPI et permet d'identifier les badges de chaque animal. La cellule de pesage avec l'amplificateur HX711 mesure le poids dans la gamelle avec assez de précision pour doser les croquettes. Le servomoteur continu entraîne la vis sans fin. Le capteur ultrasonique HC-SR04 mesure la distance entre le haut du réservoir et les croquettes pour évaluer le niveau de stock. Le capteur PIR détecte la présence d'un animal devant le distributeur, et la LED RGB fournit un retour visuel sur l'état du système.

2. Schéma de câblage avec Fritzing

J'ai réalisé le schéma de câblage complet du distributeur sur Fritzing. Ce logiciel permet de représenter visuellement les connexions entre les composants et de générer les fichiers nécessaires pour la fabrication du PCB.

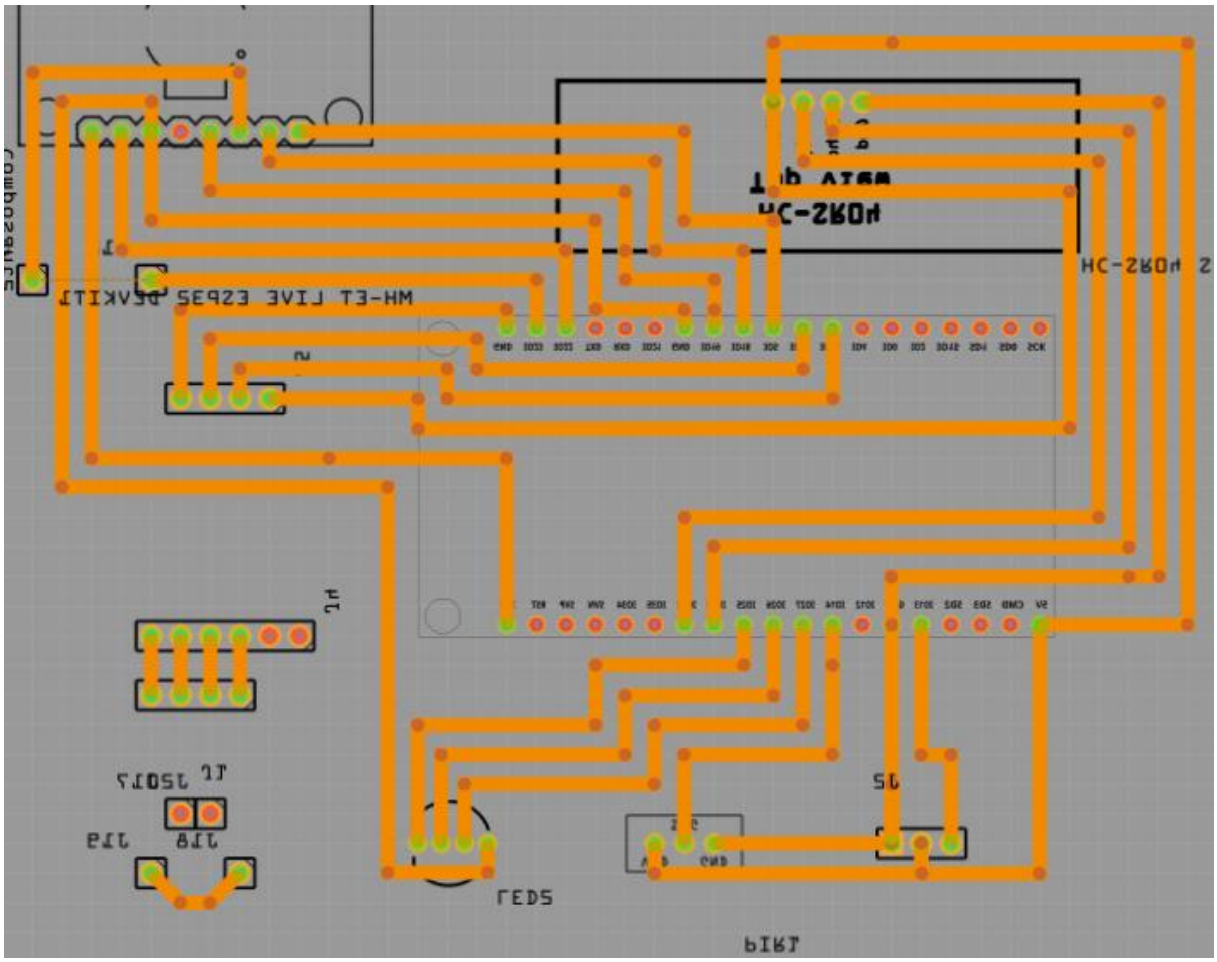

Figure 8 : Routage du PCB sous Fritzing

3. Brochage de l'ESP32

Le brochage de l'ESP32 est réparti de la manière suivante. Le HX711 utilise les GPIO 16 et 17. Le capteur ultrasonique utilise les GPIO 32 et 33. Le servomoteur est connecté au GPIO 13. Le capteur PIR au GPIO 14. Le lecteur RFID utilise les GPIO 5 et 22 avec le bus SPI par défaut sur les GPIO 18, 19 et 23. La LED RGB utilise les GPIO 25, 26 et 27 pour les trois couleurs. Les pins SPI ne sont pas déclarées explicitement dans le code car elles sont utilisées automatiquement par la bibliothèque SPI.

4. Fabrication du PCB

a) Logiciels utilisés : FlatCAM et Candle

Pour la fabrication du PCB, j'ai exporté les fichiers Gerber depuis Fritzing, puis j'ai utilisé FlatCAM pour générer le G-code à partir de ces fichiers. FlatCAM convertit les tracés du PCB en instructions de gravure pour la CNC. Une fois le G-code généré, j'ai utilisé Candle pour envoyer les instructions à la CNC et contrôler la gravure en temps réel.

b) La CNC 3018 3 axes

La gravure a été réalisée sur une CNC 3018 3 axes. C'est une machine de gravure qui utilise une fraise pour retirer le cuivre d'une plaque de circuit imprimé. Le PCB était une simple face. La machine grave les pistes en enlevant le cuivre autour des tracés, laissant uniquement les pistes conductrices.

Figure 9 : CNC 3018 de La Fabrique Connectée

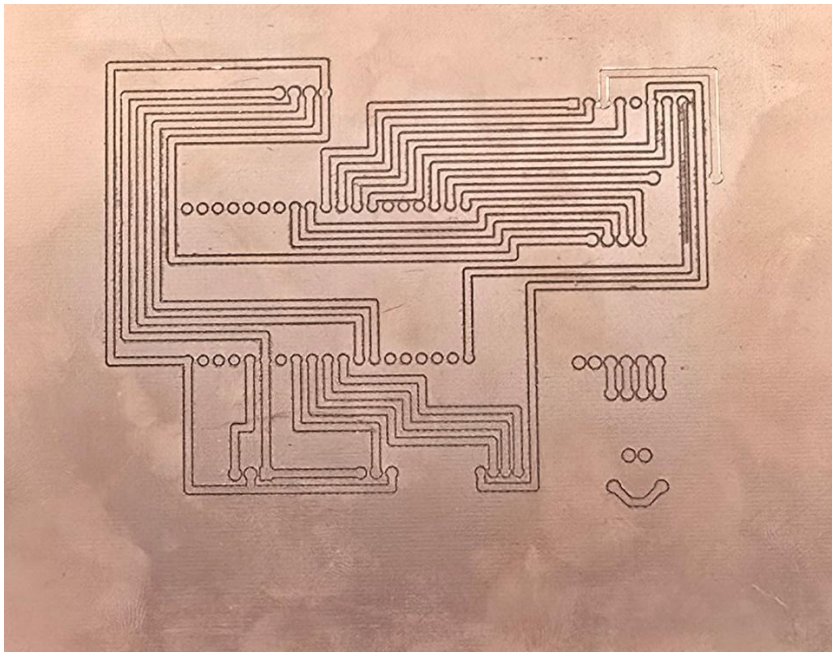

Figure 10 : PCB avant le crash

c) Problèmes rencontrés

La fabrication du PCB a été la partie la plus problématique du projet. Le logiciel Candle a perdu la position des axes X et Y en plein milieu de la gravure, ce qui a décalé toutes les pistes. A plusieurs reprise le mapping était incorrecte et donc certaines pistes ne se graver pas. Après plusieurs essais j'ai donc préféré laisser les plaques de cuivre aux autres élèves pour leurs projet. Le schéma du PCB est néanmoins finalisé sur Fritzing et prêt à être gravé. Le distributeur fonctionne donc actuellement avec un câblage sur breadboard.

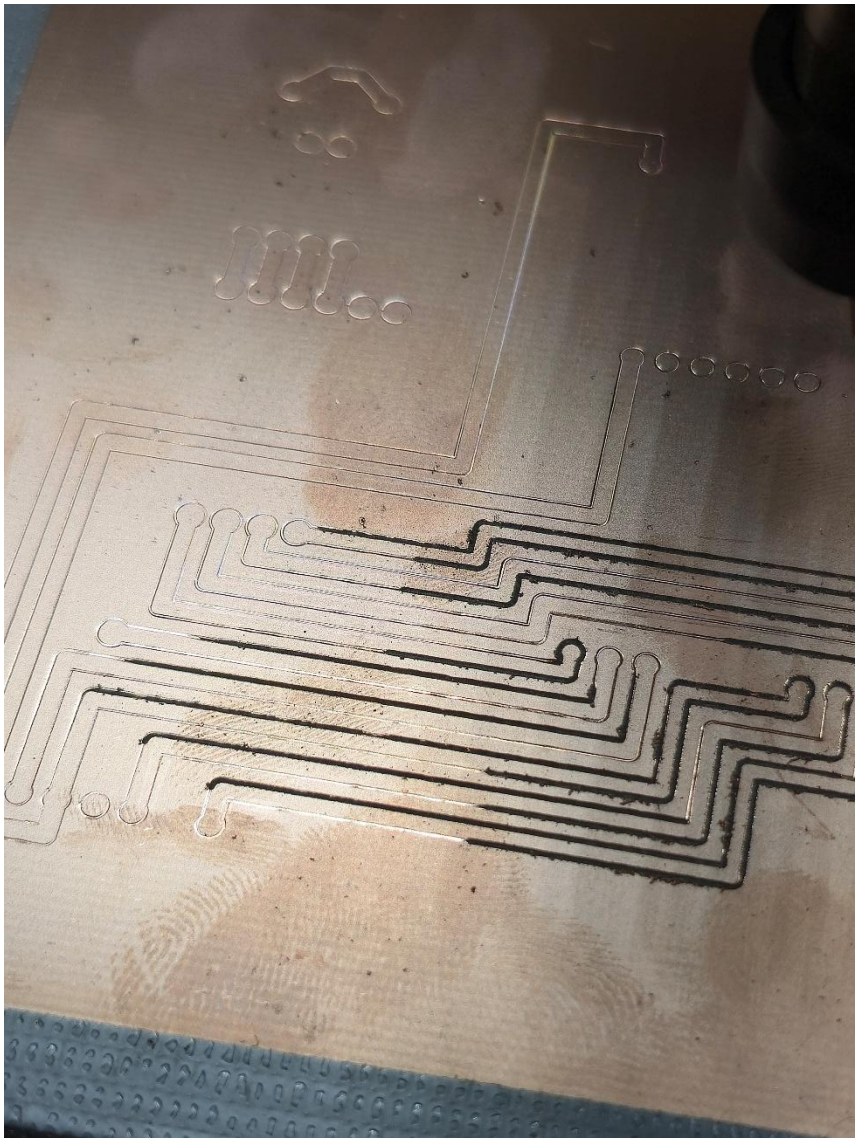

Figure 11 : Plaque PCB après un essai de gravure raté

VI. Programmation ESP32

1. Architecture logicielle

a) Structure du code

Le code de l'ESP32 est organisé en deux fichiers principaux. Le fichier config.h contient toutes les constantes du projet : les définitions de pins, les limites du système, les paramètres de distribution, les UUIDs BLE, et les structures de données. Le fichier main.cpp contient le programme, organisée en plusieurs bloc : gestion BLE, commandes, distribution, capteurs, communication réseau, persistance en mémoire NVS. L'ensemble du code est disponible sur le dépôt GitHub du projet en annexe.

b) Bibliothèques utilisées

Le projet utilise plusieurs bibliothèques : NimBLE v2 pour la communication Bluetooth Low Energy, ArduinoJson v7 pour le traitement des messages JSON, HX711 pour la lecture de la cellule de pesage, MFRC522 pour le lecteur RFID, ESP32Servo pour le contrôle du servomoteur, et PubSubClient pour la communication MQTT.

2. Communication BLE

a) Protocole JSON

La communication entre l'application mobile et l'ESP32 se fait en Bluetooth Low Energy via un protocole JSON. L'ESP32 expose un service BLE qui supporte l'écriture et les notifications. L'application envoie des commandes sous forme de messages JSON et l'ESP32 répond sur la même caractéristique via des notifications. Cela permet d'avoir les messages en clair dans le moniteur série ce qui est très pratique pour débugger l'ensemble.

b) Commandes implémentées

L'ESP32 supporte les commandes suivantes : get_all pour récupérer la liste des animaux et les statistiques du système en une seule requête, add_animal pour ajouter un nouvel animal avec ses paramètres, delete_animal pour supprimer un animal, feed_now pour forcer une distribution en ignorant le temps d'attente, tare_scale pour tarer la balance, set_wifi pour configurer le WiFi, et set_mqtt pour configurer le serveur MQTT.

c) Fragmentation des messages

Le BLE a une limitation de taille par notification. Pour les réponses volumineuses comme la liste des animaux, le message JSON est envoyé en plusieurs fragments. Côté application mobile, un buffer accumule les fragments reçus et tente de lire le JSON après chaque réception. Dès que le JSON est complet et valide, il est traité.

Pour éviter ce problème, j'ai réduit la taille des clés JSON dans la réponse get_all, ce qui permet de faire tenir les données de 3 animaux et les statistiques dans un seul paquet de moins de 500 octets. Il est possible de s'éparer les réponses en plusieurs paquet mais de mon côté je n'ai pas réussi à le faire.

3. Distribution contrôlée par pesée

a) Principe de fonctionnement

Quand un badge RFID valide est scanné et que le temps d'attente est respecté, le distributeur lance une distribution. Le servomoteur démarre et fait tourner la vis sans fin. Pendant que la vis tourne, la balance mesure le poids dans la gamelle toutes les 150 millisecondes. Dès que le poids atteint 90% de la ration demandée (ou la valeur définie dans la config DISTRIB_STOP_PERCENT), le servomoteur s'arrête. Les pourcent restants sont complétés par l'inertie des croquettes encore en train de tomber. Après un temps de stabilisation de 1,5 seconde, une mesure finale précise est effectuée.

b) Calibration de la balance

La cellule de pesage demande une calibration pour convertir les valeurs brutes du HX711 en grammes. J'ai écrit un programme de test qui affiche les valeurs de la balance en temps réel et j'ai utilisé un objet d'un poids connu dans mon cas mon téléphone pour calculer le facteur de calibration qui est de 1003.97. Ce facteur est défini dans le fichier config.h et peut être ajusté facilement. La tare de la balance peut aussi être faite depuis l'application mobile.

c) Seuil d'arrêt et confirmations anti-vibrations

Un problème que j'ai rencontré est que les croquettes qui tombent dans la gamelle créent des vibrations qui faussent les mesures de la balance. Une seule lecture peut donner un pic très au-dessus du poids réel, ce qui déclenche un arrêt trop tôt. Pour résoudre ce problème, j'ai mis en place deux choses : chaque mesure est une moyenne sur 3 lectures du HX711, et il faut 3 mesures d'affilée au-dessus du seuil de 90% avant d'arrêter le servomoteur. Cela élimine les faux positifs causés par les vibrations.

d) Problèmes de précision et solutions

Lors des premiers tests, le distributeur distribuait trop de nourriture à chaque fois. Le servomoteur tournait à vitesse maximale et la balance ne mesurait que toutes les 200 millisecondes. Pendant ce laps de temps, la vis déversait déjà beaucoup trop de croquettes. J'ai réduit la vitesse du servomoteur et augmenté la fréquence de pesée. Le seuil d'arrêt à 90% au lieu de 100% permet aussi de compenser l'inertie. L'ensemble de ces paramètres est configurable dans le fichier config.h.

4. Identification RFID

Le lecteur RFID MFRC522 permet d'identifier les animaux grâce à des badges individuels. Quand un badge est scanné, l'ESP32 compare l'UID lu avec ceux enregistrés pour chaque animal. Si le badge correspond à un animal et que le temps d'attente est respecté, la distribution est lancée. L'association entre un animal et son badge se fait depuis l'application mobile. La vérification du RFID se fait toutes les 100 millisecondes dans la boucle principale.

5. Mesure du niveau de stock

Le capteur ultrasonique HC-SR04 est positionné au-dessus du réservoir et mesure la distance entre le capteur et les croquettes. Plus la distance est petite, plus le réservoir est rempli. La hauteur du réservoir est de 13,4 cm, ce qui correspond à 0% quand il est vide et 100% quand il est plein. Pour obtenir des mesures correctes malgré la surface irrégulière des croquettes, le code effectue 5 mesures et prend la médiane.

6. Détection de présence

Le capteur PIR détecte la présence d'un animal devant le distributeur. Quand une présence est détectée, le système commence à chercher un badge. Ce système évite de lire le RFID en permanence quand aucun animal n'est présent. Même si le distributeur est branché sur secteur cela permet de faire des économies d'énergie.

7. Retour visuel LED

La LED RGB indique l'état du distributeur : bleu quand un appareil BLE est connecté, jaune quand une présence est détectée, vert quand un badge est reconnu, vert clignotant pendant la distribution, rouge fixe quand le temps d'attente n'est pas respecté, et rouge clignotant en cas d'erreur. Je ne l'ai pas du tout tester dans la version finale car je ne trouver pas cela très pertinent de l'ajouter mais je l'ai tout de même inclus dans le code elle devrait fonctionner.

8. Communication réseau

a) WiFi et reconnexion automatique

L'ESP32 se connecte au WiFi au démarrage si un SSID est configuré. Si la connexion échoue, le système tente une reconnexion automatique toutes les 30 secondes. Le WiFi et le MQTT ne sont pas indispensables : le distributeur fonctionne normalement sans connexion réseau, il distribue la nourriture et gère les temps d'attente en local. La connexion réseau ne sert qu'à remonter les données vers le serveur.

b) MQTT : publication périodique et topics

Quand le WiFi est connecté, l'ESP32 publie des messages MQTT sur plusieurs topics. Le topic feeder/bowl_weight est publié toutes les 60 secondes et contient le poids dans

la gamelle et le niveau de stock. Le topic feeder/rfid_scan est publié à chaque scan de badge et contient le nom de l'animal et l'horodatage. Le topic feeder/distributed est publié à chaque fin de distribution et contient les détails de la distribution. Tous les messages contiennent un horodatage.

▼ 10.0.0.191
▼ feeder
status = online
bowl_weight = {"weight":0.0,"stock":0,"ts":"2026-03-17T15:02:20"}
rfid_scan = {"animal":"Blanc","uid":"B3A79018","ts":"2026-03-17T14:49:56"}
distributed = {"animal":"Blanc","requested":47,"distributed":0.0,"bowl":0.0,"feeds":23,"ts":"2026-03-17T14:50:11"}

Figure 12 : Messages MQTT reçus dans MQTT Explorer

VII. Application mobile Flutter

1. Technologies et architecture

L'application mobile a été développée en Flutter avec le langage Dart, pour la plateforme Android. Flutter est un framework de Google qui permet de créer des applications mobiles avec un seul code source. La communication BLE est gérée par la bibliothèque flutter_blue_plus. L'application est organisée en 4 pages et un helper pour les calculs de ration. Je n'ai jamais fait de développement mobile et étant donné que je ne le maîtrisais pas, je me suis appuyé sur des templates existants pour la communication Bluetooth et de l'IA pour résoudre certains bugs spécifiques à Flutter et au BLE. C'était une expérience intéressante même si j'ai pris beaucoup de temps à réaliser l'application. Au total j'ai réalisé 27 versions de cette dernière.

L'APK est déjà compilée et prête à installer sur Android. Pour iOS, Flutter permet de compiler le même code pour iPhone mais cela nécessite un Mac avec Xcode. Le portage devrait normalement être très simple car tout le code est déjà compatible, il suffit de lancer la compilation depuis un Mac c'est l'avantage de Futter.

2. Page Scan BLE

La première page de l'application permet de scanner les appareils Bluetooth à proximité. Les résultats sont filtrés pour n'afficher que les appareils dont le nom contient « Feeder ». Quand l'utilisateur clique sur un appareil, l'application se connecte, découvre les services BLE, et cherche la caractéristique qui supporte l'écriture et les notifications. Si la connexion réussit, l'utilisateur est redirigé vers la page des animaux. Sur android il faut laisser à l'application les autorisations suivantes : position et Bluetooth.

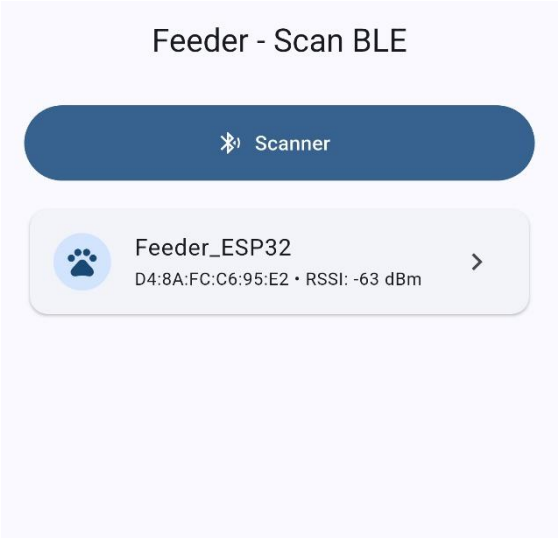

Figure 14 : Page Scan BLE

3. Page Liste des animaux

a) Requête unique get_all

À l'ouverture de la page, l'application envoie une seule commande get_all qui retourne à la fois la liste des animaux et les statistiques du système. Cette approche en une seule requête évite les problèmes qui se produisaient quand deux requêtes étaient envoyées l'une après l'autre (lorsque plusieurs paquet était envoyer en même temps).

b) Barre de statut

En haut de la page, une barre de statut affiche le niveau de stock en pourcentage, le poids dans la gamelle, l'état de la connexion WiFi, et l'état de la connexion MQTT. Chaque animal est affiché sur une carte avec son nom, son type, son âge, son poids, sa ration, son temps d'attente, le nombre de repas distribués et un bouton pour distribuer la nourriture. Un dialogue de confirmation s'affiche avant de lancer la distribution forcée pour éviter les erreurs.

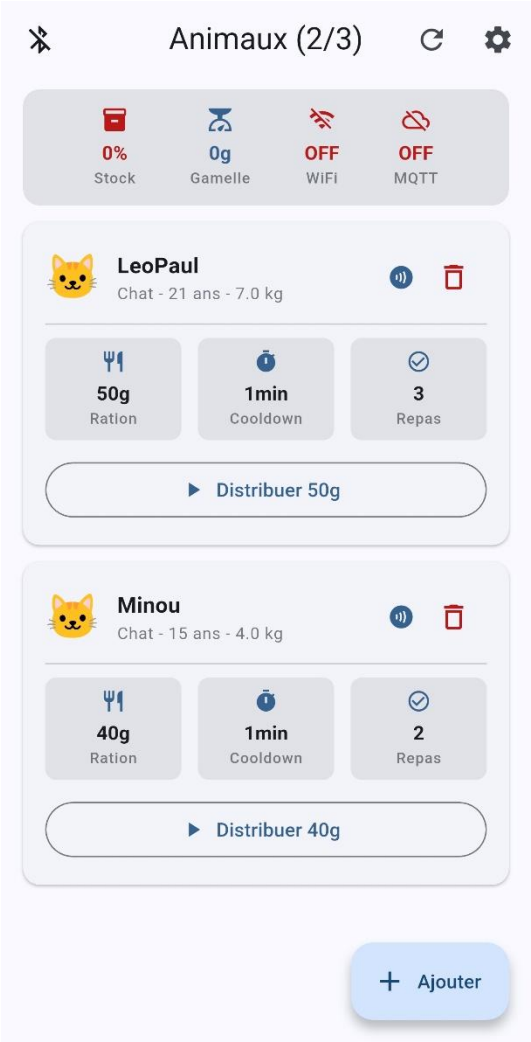

Figure 15 : Page Liste des animaux

4. Page Ajout animal

a) Formulaire avec recommandations automatiques

La page d'ajout contient un formulaire avec le nom de l'animal, son type, son âge, et son poids. Quand le mode recommandations automatiques est activé, l'application calcule automatiquement la ration et le temps d'attente en fonction du type, de l'âge et du poids de l'animal. Les calculs sont basés sur des pourcentages du poids corporel adaptés à chaque catégorie d'âge.

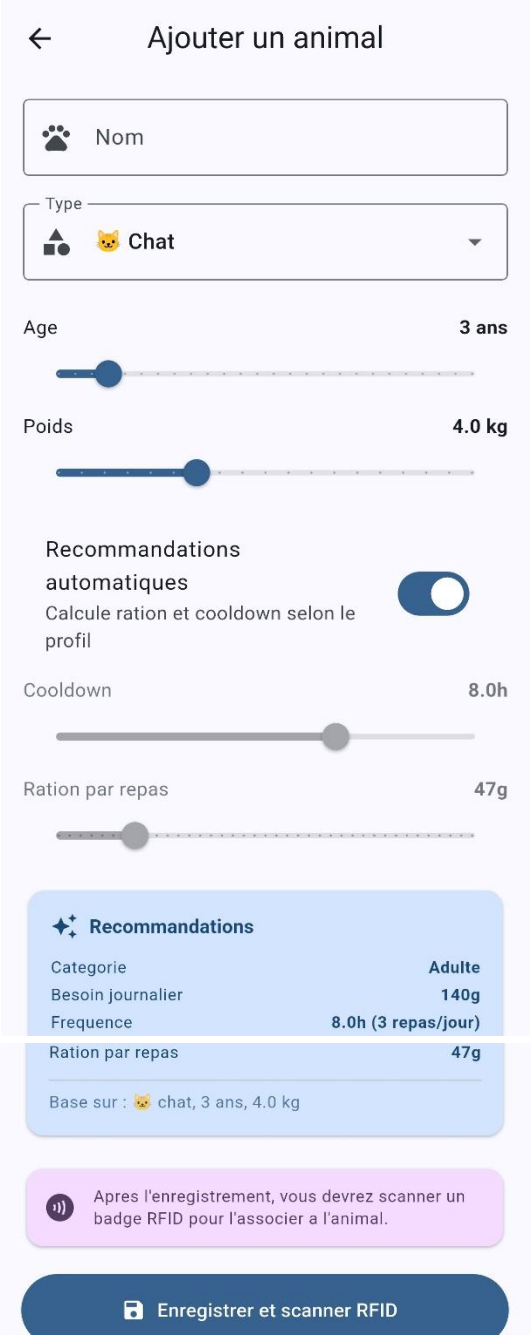

Figure 16 : Page Ajout animal

b) Association du badge RFID

Après avoir validé le formulaire, l'application affiche un écran de scan RFID avec une animation et un compte à rebours de 60 secondes. L'utilisateur doit présenter un badge RFID devant le lecteur du distributeur. Quand le badge est détecté, l'ESP32 l'associe à l'animal et envoie une confirmation. L'utilisateur peut aussi passer cette étape s'il ne souhaite pas associer de badge tout de suite. Cela ne sert à rien car je n'ai pas eu le temps d'ajouter une option pour modifier le badge RFID.

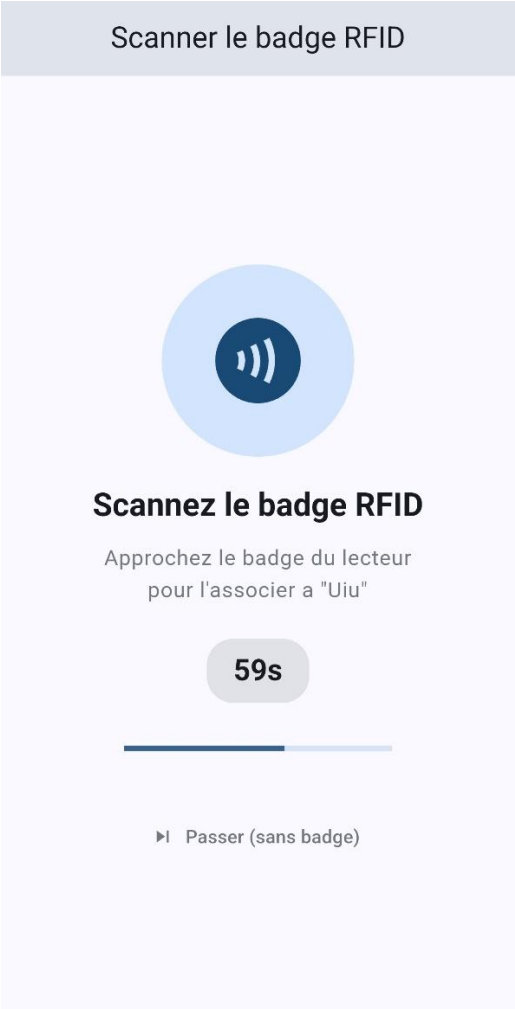

Figure 17 : Page Scan RFID

5. Page Paramètres

La page de paramètres permet de configurer la connexion WiFi avec le SSID et le mot de passe, la connexion MQTT avec l'adresse du serveur et le port, et de tarer la balance. Les paramètres sont envoyés à l'ESP32 qui les sauvegarde en mémoire NVS pour les conserver après un redémarrage.

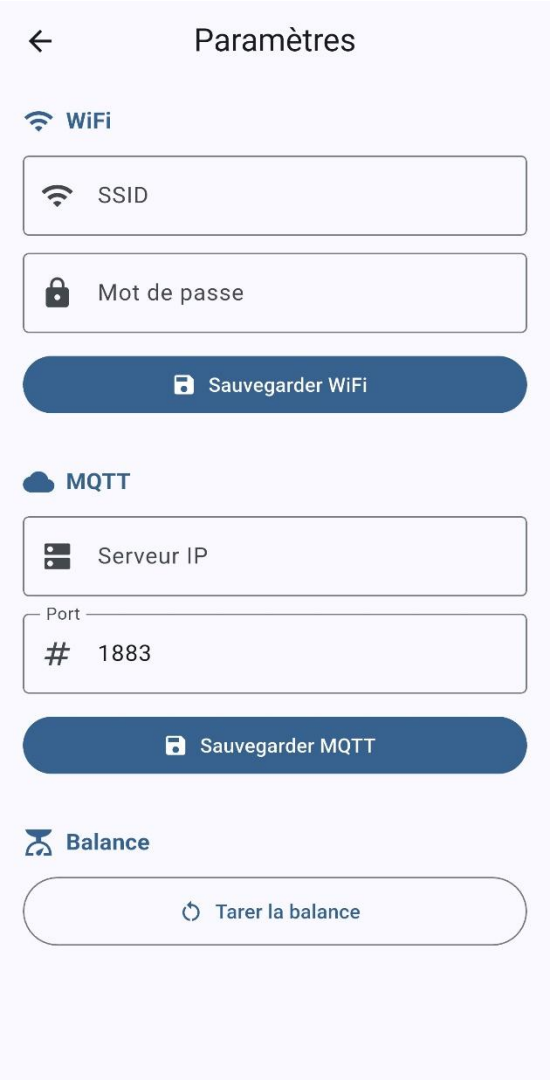

Figure 18 : Page Paramètres

VIII. Infrastructure serveur

1. Architecture globale

L'infrastructure serveur est entièrement hébergée sur le Raspberry Pi 4. L'ESP32 envoie les données via WiFi au broker MQTT Mosquitto. Node-RED écoute les topics MQTT et insère les données dans la base MariaDB. Grafana lit les données dans MariaDB et les affiche sur un dashboard. L'ensemble forme une chaîne de collecte, stockage et visualisation des données.

2. Mosquitto

Mosquitto est le broker MQTT installé sur le Raspberry Pi 4. Il reçoit les messages publiés par l'ESP32 et les redistribue aux clients abonnés. La configuration est simple : le broker écoute sur le port 1883 et accepte les connexions anonymes.

3. Node-RED

Node-RED fait le pont entre MQTT et la base de données. C'est un outil de programmation qui fonctionne sur navigateur. Au lieu d'avoir à écrire un script, on relie des blocs entre eux par des fils. J'ai créé un flow qui s'abonne aux 4 topics MQTT du distributeur. Pour chaque message reçu, une fonction JavaScript extrait les données du JSON et génère une requête INSERT SQL. Le résultat est envoyé à un noeud MySQL qui exécute la requête dans MariaDB. L'avantage de Node-RED est que tout se configure visuellement sans écrire de code serveur.

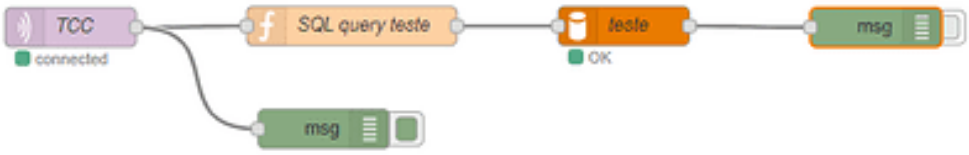

Figure 19 : Exemple d'un Flow Node-RED

4. MariaDB

MariaDB stocke les données dans 4 tables : feed_events pour les distributions, rfid_scans pour les passages de badge, bowl_weight pour l'historique du poids de gamelle et du niveau de stock, et errors pour les erreurs système. Chaque table contient un horodatage et des index pour accélérer les requêtes Grafana.

5. Grafana

Grafana est l'outil de visualisation qui affiche les données sous forme de dashboard. J'ai configuré plusieurs panneaux : le dernier passage de chaque animal, une alerte si un animal n'a pas été vu depuis une durée configurable, une jauge du niveau de stock avec un code couleur vert au-dessus de 25%, orange entre 10% et 25%, rouge en dessous de 10%, et le poids dans la gamelle en temps réel. Le dashboard se met à jour automatiquement toutes les 30 secondes.

Figure 20 : Dashboard Grafana

IX. Problèmes rencontrés et solutions

Cette section regroupe les principaux problèmes techniques rencontrés au cours de la SAE et les solutions que j'ai mises en place pour les résoudre.

1. BLE invisible au scan

À plusieurs reprises pendant le développement, l'ESP32 n'était plus visible lors du scan BLE depuis l'application mobile et même depuis des outils de test comme nRF Connect. Le moniteur série affichait pourtant que le BLE était bien démarré.

La première cause était la connexion MQTT bloquante. Quand le serveur MQTT était injoignable avec une mauvaise adresse IP, la fonction de connexion bloquait le processeur pendant 15 secondes à chaque tentative. Pendant ce temps, le BLE ne répondait pas. La solution a été de réduire le timeout à 2 secondes et d'espacer les tentatives à 30 secondes.

La deuxième cause était le NVS corrompu. Après de nombreux reflash, la mémoire persistante de l'ESP32 contenait des données incohérentes qui empêchaient le BLE de s'initialiser correctement. La solution a été d'effacer complètement la flash avec la commande Erase Flash dans PlatformIO avant de reflasher le code.

2. Surdosage lors de la distribution

Lors des premiers tests, le distributeur distribuait systématiquement trop de nourriture, parfois le double de la quantité demandée. Le servomoteur tournait à vitesse maximale et la balance ne mesurait que toutes les 200 millisecondes. Pendant ce temps, la vis déversait beaucoup trop de croquettes. De plus, une seule mesure de la balance suffisait pour arrêter le moteur, et les vibrations pouvaient donner un faux pic de poids.

La solution combine plusieurs ajustements : réduction de la vitesse du servomoteur, augmentation de la fréquence de pesée, arrêt à 90% de la cible pour compenser l'inertie, et exigence de 3 mesures d'affilée au-dessus du seuil avant d'arrêter. L'ensemble des paramètres est configurable dans config.h.

3. La vis sans fin : 4 impressions

La vis sans fin a été la pièce la plus difficile à imprimer. Le premier essai en PETG a donné des supports impossibles à enlever. Le deuxième en PLA avait des supports difficiles et une vis trop fine. Le troisième avec du PLA support était bien imprimé mais les creux étaient trop réguliers et les croquettes se bloquaient. Le quatrième avec des creux plus espacés et du PLA support a donné le résultat attendu.

4. Crash de la CNC

La fabrication du PCB a échoué à cause de crashs de la CNC. Le logiciel Candle a perdu la position des axes X et Y en plein milieu de la gravure, décalant toutes les

pistes. N'ayant qu'une grande plaque disponible, j'ai préféré la laisser aux autres élèves. Le schéma PCB reste finalisé et prêt à être gravé.

5. Listeners BLE concurrents dans Flutter

Quand l'utilisateur ouvrait la page d'ajout par-dessus la page des animaux, les deux pages écoutaient les notifications BLE en même temps. La page des animaux interceptait les réponses destinées à la page d'ajout, ce qui empêchait l'association RFID de fonctionner. La solution a été de mettre en pause le listener de la page des animaux quand on navigue vers une autre page, et de le réactiver au retour.

6. Filtre de scan BLE invisible sur Android

L'application ne détectait plus le distributeur lors du scan. Le filtre utilisait le champ platformName pour identifier les appareils, mais sur certains téléphones Android ce champ est vide. Le vrai nom se trouve dans les données d'advertising. La solution a été de vérifier les deux champs : advName en priorité puis platformName en secours.

Conclusion

Ce projet m’a permis de concevoir tout un projet de bout en bout, de la modélisation à la conception. Le distributeur automatique est fonctionnel : il identifie les animaux, distribue la bonne quantité de nourriture grâce à la pesée en temps réel, respecte les temps d’attente, et remonte les données vers le rpi serveur pour le suivi.

J'ai beaucoup aimé réaliser ce projet. Le fait que chaque élève ait son propre sujet à développer, tout en étant relié à un Raspberry Pi central commun, rend la SAE très intéressante. On travaille chacun de son côté sur un vrai projet complet, mais on partage quand même une infrastructure commune. C'est une approche que j'ai trouvée plus pertinente que de travailler en groupe sur un seul sujet.

Un des points qui je pense mériterait d'être amélioré dans la SAE est la partie CNC. On a eu très peu de plaques de cuivre à disposition, ce qui limitait fortement le nombre de tests possibles. (je sais que tout cela est liée au budget) Une séance un peu plus grande dédiée à la CNC serait utile, avec par exemple des conseils sur Fritzing. Par défaut les pistes sont trop fines et ressortent très mal à la gravure, voire pas du tout et on ne s’en aperçoit qu’après gravure. Savoir qu'on peut augmenter leur taille dans les paramètres de Fritzing avant d'exporter les Gerber nous aurait fait gagner beaucoup de temps. Avoir un peu plus d'accompagnement que juste le fonctionnement de base de la machine serait un vrai plus pour les prochaines promotions.

Plusieurs améliorations sont possibles. La fabrication du PCB n'a pas pu être finalisée à cause des problèmes de CNC, mais le schéma est prêt. L'utilisation de matériaux certifiés alimentaires rendrait le projet utilisable en conditions réelles. Un système de notification push pourrait alerter l'utilisateur quand le stock est bas ou quand un animal n'a pas été vu depuis longtemps. Enfin, le support de plus d'animaux pourrait être ajouté en optimisant davantage le protocole BLE.

Merci à vous M. Jeanneret pour ses cours d'IOM. On voit que c'est un sujet qui vous tient énormément à cœur et c'est plaisant de vous écouter en parler.

Glossaire

BLE : Bluetooth Low Energy, version basse consommation du Bluetooth utilisée pour la communication sans fil courte portée.

CNC : Computer Numerical Control, machine à commande numérique utilisée pour la gravure du PCB.

ESP32 : Microcontrôleur avec WiFi et Bluetooth intégrés, fabriqué par Espressif Systems.

Grafana : Outil de visualisation de données sous forme de dashboards.

HX711 : Amplificateur de signal pour cellule de pesage.

JSON : JavaScript Object Notation, format de données textuelles utilisé pour la communication.

MariaDB : Système de gestion de base de données relationnelle.

MFRC522 : Module lecteur RFID communicant en SPI.

MQTT : Message Queuing Telemetry Transport, protocole de messagerie léger pour l'IoT.

Node-RED : Outil de programmation visuelle pour l'IoT, fonctionnant dans le navigateur.

NVS : Non-Volatile Storage, mémoire persistante de l'ESP32 pour sauvegarder les paramètres.

PCB : Printed Circuit Board, circuit imprimé.

PLA : Polylactic Acid, filament d'impression 3D à base de fécule de maïs.

RFID : Radio-Frequency Identification, technologie d'identification par radio-fréquence.

SPI : Serial Peripheral Interface, protocole de communication série.

TPU : Thermoplastic Polyurethane, filament souple pour l'impression 3D.

Bibliographie

Espressif Systems, documentation ESP32 : https://docs.espressif.com/

NimBLE-Arduino, bibliothèque BLE pour ESP32 :
https://github.com/h2zero/NimBLE-Arduino

ArduinoJson v7, documentation : https://arduinojson.org/

Flutter, documentation officielle : https://flutter.dev/docs

flutter_blue_plus, bibliothèque BLE pour Flutter :
https://pub.dev/packages/flutter_blue_plus

PubSubClient, bibliothèque MQTT pour Arduino :
https://github.com/knolleary/pubsubclient/

Node-RED, outil de programmation visuelle : https://nodered.org/

Système de verrouillage à crans, tutoriel YouTube : https://youtu.be/fGv8D-E9sgI

Annexes

A. Notice d'utilisation

1. Brancher le distributeur en micro-USB.
2. Ouvrir l'application Feeder sur le téléphone Android.
3. Appuyer sur Scanner et sélectionner Feeder_ESP32 dans la liste.
4. Sur la page des animaux, appuyer sur + pour ajouter un animal.
5. Remplir le formulaire et appuyer sur Enregistrer.
6. Présenter un badge RFID devant le lecteur dans les 60 secondes.
7. L'animal est enregistré. Quand il s'approche du distributeur et présente son badge, la distribution se lance automatiquement.
8. Pour configurer le WiFi et le MQTT, aller dans la page Paramètres.

B. Compilation de l'application mobile

L'APK Android est déjà compilée et disponible dans le dépôt GitHub. Pour recompiler l'application :

1. Installer Flutter et le SDK Android.
2. Ouvrir le projet dans un terminal.
3. Lancer flutter pub get pour installer les dépendances.
4. Lancer flutter build apk --release pour compiler.
5. L'APK se trouve dans build/app/outputs/flutter-apk/app-release.apk.
6. Copier l'APK sur le téléphone et lancer l'installation.

Pour iOS, il suffit d'avoir un Mac avec Xcode installé et de lancer flutter build ios. Le code Flutter est déjà compatible, aucune modification n'est nécessaire. Flutter génère un seul code source pour les deux plateformes, ce qui rend le portage très simple.

En cas d'erreur « SDK 36 not found » : ouvrir Android Studio, aller dans Tools, SDK Manager, cocher Android 15.0 API 36 et appliquer.

C. Notice de mise à jour et dépannage

Pour mettre à jour le code de l'ESP32 :

1. Ouvrir le projet dans VS Code avec l'extension PlatformIO.
2. Modifier les fichiers src/main.cpp ou include/config.h selon les besoins.
3. Connecter l'ESP32 en USB et cliquer sur Upload dans PlatformIO.

Si le BLE ne fonctionne plus après une mise à jour, il faut effacer la mémoire NVS de l'ESP32. Le NVS conserve les anciens paramètres WiFi, MQTT et les animaux enregistrés. Si ces données sont corrompues, le BLE peut ne plus s'initialiser correctement. Pour résoudre ce problème :

1. Dans VS Code, cliquer sur l'icône PlatformIO dans la barre latérale gauche.
2. Déplier esp32dev, puis Platform.
3. Cliquer sur Erase Flash. Attendre que l'opération se termine.
4. Cliquer sur Upload pour reflasher le code.

Attention : l'effacement du NVS supprime tous les paramètres sauvegardés. Il faudra reconfigurer le WiFi, le MQTT et les animaux depuis l'application mobile après le reflash.

D. Lien GitHub

L'ensemble du code source du projet est disponible sur GitHub :

https://github.com/RemiXNava98/feeder-SAE6-IOM-01

Le dépôt contient le code ESP32, le code de l'application Flutter, les fichiers de configuration Node-RED, le schéma SQL de la base de données, et le schéma Fritzing du PCB.

Table des illustrations

Figure 1 : Distributeur assemblé complet

Figure 2 : Distributeur avec des Skittles pour les tests

Figure 3 : Bloc électronique ouvert

Figure 4 : Modèle 3D de la vis sans fin

Figure 5 : Vue intérieure du stockage et de la vis sans fin

Figure 6 : Colonne avec le capteur ultrason

Figure 7 : Partie pesée avec la cellule de charge

Figure 8 : Routage du PCB sous Fritzing

Figure 9 : CNC 3018 de La Fabrique Connectée

Figure 10 : PCB en cours de gravure

Figure 11 : Plaque PCB après un essai raté

Figure 12 : Messages MQTT dans MQTT Explorer

Figure 14 : Page Scan BLE

Figure 15 : Page Liste des animaux

Figure 16 : Page Ajout animal

Figure 17 : Page Scan RFID

Figure 18 : Page Paramètres

Figure 19 : Flow Node-RED

Figure 20 : Dashboard Grafana